

Brush

Brush

1. Course Content
2. Program Startup
3. Core Code Analysis

1. Course Content

- Run example programs using the mediapipe framework to detect fingers, then select colors, and draw movement tracks on the image based on the detected finger movements.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

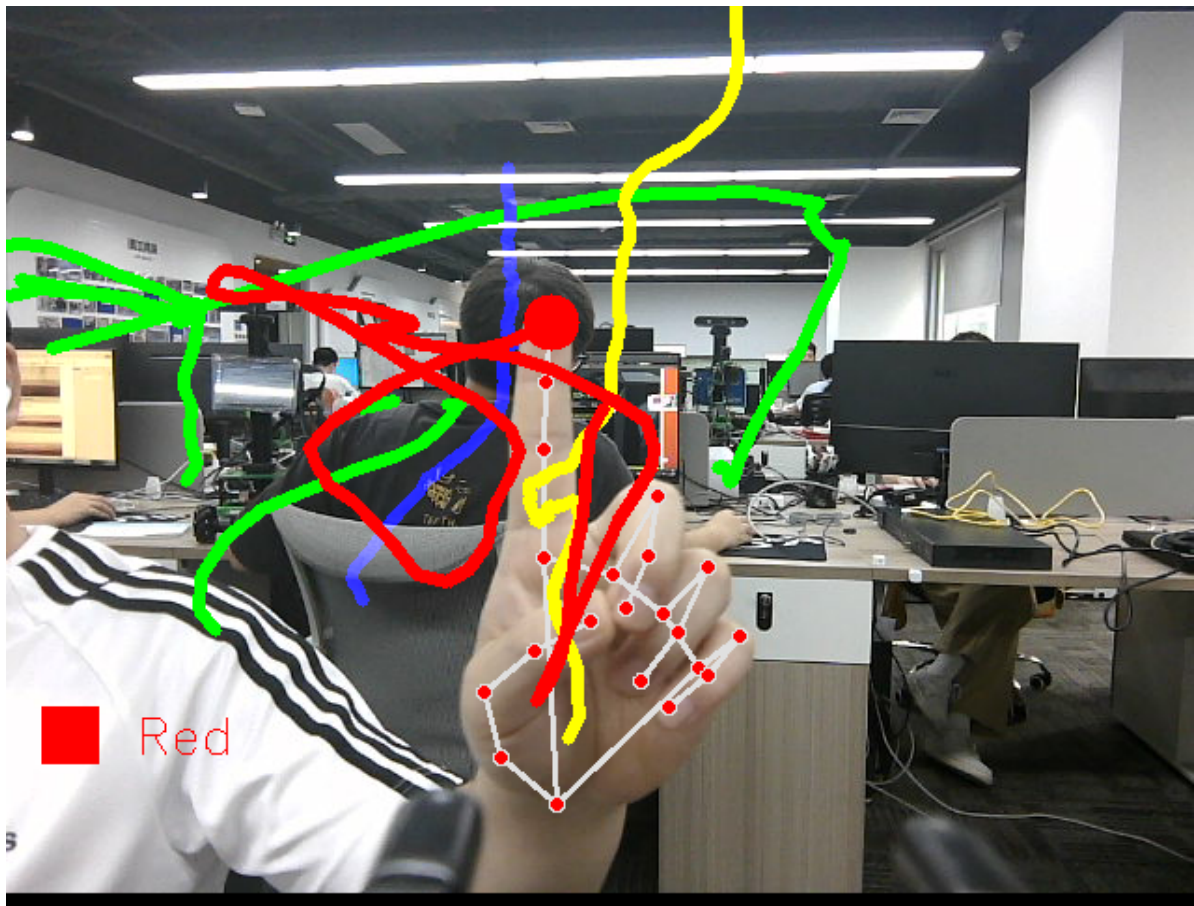
```
exec_m3
```

- Run brush example program in terminal

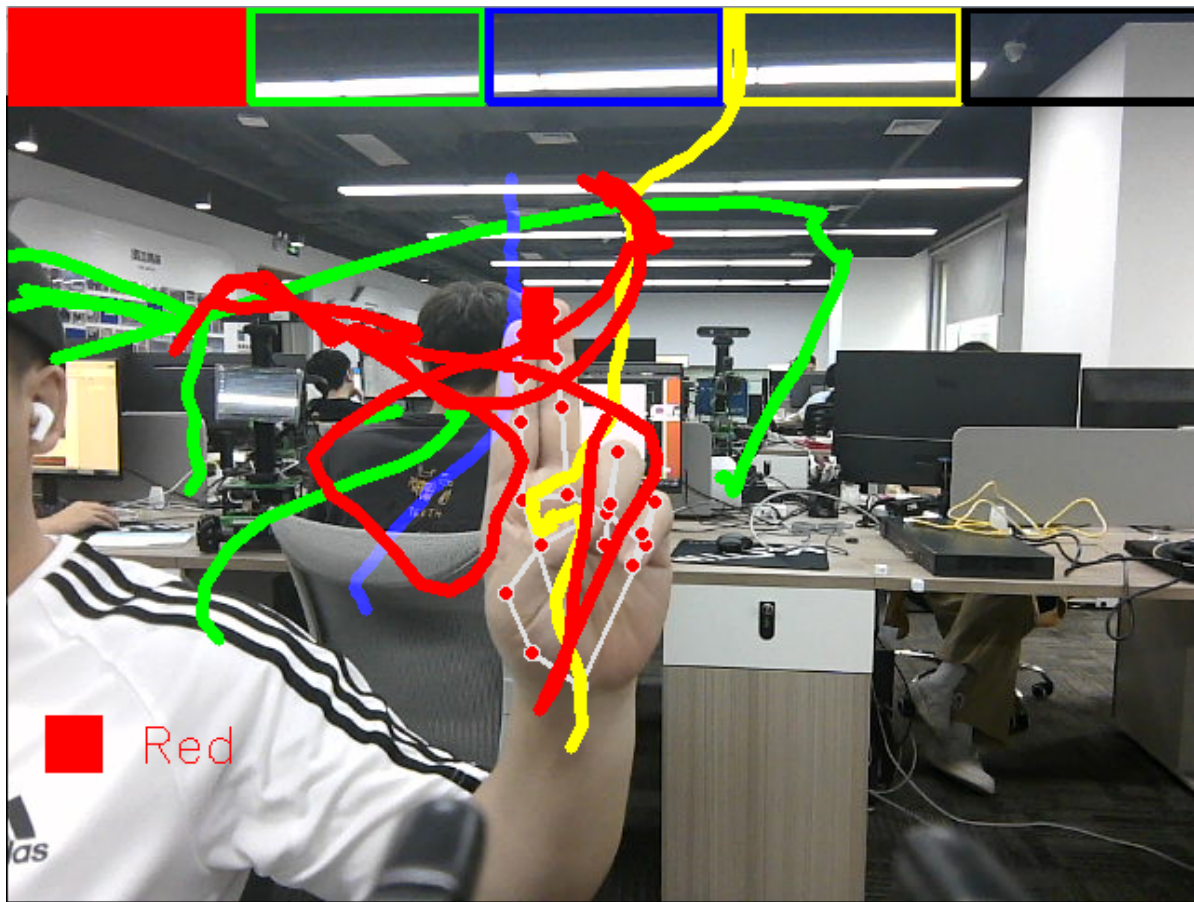
```
ros2 launch m3_bringup mediapipe_demo.launch.py demo:=VirtualPaint
```

After the program runs, as shown in the figure below, the default fingertip color is red. When the right index finger and middle finger are together, it is in selection mode, and a color selection box appears. When the two fingertips move to the corresponding color position, that color is selected (black is the eraser); when the index finger and middle finger are separated, it is in drawing mode, and you can draw freely on the canvas.

Index finger and middle finger separated to enter drawing mode,



Index finger and middle finger together to enter color selection mode,



3. Core Code Analysis

Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/mediapipe/08_VirtualPaint.py
```

- Library files used,

```
import math
import time
import cv2 as cv
import numpy as np
import mediapipe as mp
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2
print("import done")
```

Initialize data and define publishers and subscribers,

```
def __init__(self, name):
    super().__init__(name)
    self.xp = self.yp = self.pTime = self.boxx = 0
    self.tipIds = [4, 8, 12, 16, 20]
    #Create an empty image with resolution 480*640
    self.imgCanvas = np.zeros((480, 640, 3), np.uint8)
    #Define the thickness of the brush (red, green, blue, yellow)
    self.brushThickness = 5
    #Define the thickness of the eraser (black)
    self.eraserThickness = 100
    #Y coordinate threshold of the bottom of the color selection box, values
    smaller than this indicate entering color selection mode
    self.top_height = 50
    self.Color = "Red"
    #Define the brush color list, the following represents the RGB values of each
    color
    self.ColorList = {
        'Red': (0, 0, 255),
        'Green': (0, 255, 0),
        'Blue': (255, 0, 0),
        'Yellow': (0, 255, 255),
        'Black': (0, 0, 0),
    }
    #Define the ID of each fingertip, used later to determine which fingers are
    extended
    self.tipIds = [4, 8, 12, 16, 20]
    #Use the class in the mediapipe library to define a hand object
    self.mpHand = mp.solutions.hands
    self.mpDraw = mp.solutions.drawing_utils
    self.hands = self.mpHand.Hands(
        static_image_mode=False,
        max_num_hands=2,
        min_detection_confidence=0.85,
        min_tracking_confidence=0.5 )
```

```

#Define the properties of joint connection lines, which will be used in
subsequent joint point connection functions
self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0, 255),
thickness=-1, circle_radius=15)
self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0),
thickness=10, circle_radius=10)

self.rgb_bridge = CvBridge()
#Define the subscriber for color image topic
self.sub_rgb =
self.create_subscription(Image, "/camera/color/image_raw", self.get_RGBImageCallBa
ck, 100)

```

Color image callback function,

```

def get_RGBImageCallback(self, msg):
    #Use CvBridge to convert color image message data to image data
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Get the size of the color image for subsequent color selection based on
    fingertip coordinates
    h, w, c = rgb_image.shape
    frame, lmList = self.findHands(rgb_image, draw=False)
    key = cv2.waitKey(1)
    if len(lmList) != 0:
        # print(lmList)
        # tip of index and middle fingers
        #Get the coordinate values of the index and middle finger tips
        x1, y1 = lmList[8][1:]
        x2, y2 = lmList[12][1:]
        #Call the fingersUp function to return the list of extended fingers
        fingers = self.fingersUp()
        #If both index and middle fingers are extended
        if fingers[1] and fingers[2]:
            # print("Seclection mode")
            if y1 < self.top_height:
                #Determine the selected color based on the index finger
coordinates and image size
                if 0 < x1 < int(w / 5) - 1:
                    self.boxx = 0
                    self.Color = "Red"
                if int(w / 5) < x1 < int(w * 2 / 5) - 1:
                    self.boxx = int(w / 5)
                    self.Color = "Green"
                elif int(w * 2 / 5) < x1 < int(w * 3 / 5) - 1:
                    self.boxx = int(w * 2 / 5)
                    self.Color = "Blue"
                elif int(w * 3 / 5) < x1 < int(w * 4 / 5) - 1:
                    self.boxx = int(w * 3 / 5)
                    self.Color = "Yellow"
                elif int(w * 4 / 5) < x1 < w - 1:
                    self.boxx = int(w * 4 / 5)
                    self.Color = "Black"
                cv.rectangle(frame, (x1, y1 - 25), (x2, y2 + 25),
self.ColorList[self.Color], cv.FILLED)
                cv.rectangle(frame, (self.boxx, 0), (self.boxx + int(w / 5),
self.top_height), self.ColorList[self.Color], cv.FILLED)
                #Draw color selection boxes for each color

```

```

        cv.rectangle(frame, (0, 0), (int(w / 5) - 1, self.top_height),
self.ColorList['Red'], 3)
        cv.rectangle(frame, (int(w / 5) + 2, 0), (int(w * 2 / 5) - 1,
self.top_height), self.ColorList['Green'], 3)
        cv.rectangle(frame, (int(w * 2 / 5) + 2, 0), (int(w * 3 / 5) - 1,
self.top_height), self.ColorList['Blue'], 3)
        cv.rectangle(frame, (int(w * 3 / 5) + 2, 0), (int(w * 4 / 5) - 1,
self.top_height), self.ColorList['Yellow'], 3)
        cv.rectangle(frame, (int(w * 4 / 5) + 2, 0), (w - 1,
self.top_height), self.ColorList['Black'], 3)
        #If the index and middle fingers are not both extended and the pixel
distance between their fingertips is greater than 50, it means they are in an
open state, entering drawing mode
        if fingers[1] and fingers[2] == False and math.hypot(x2 - x1, y2 - y1) >
50:
            # print("Drawing mode")
            if self.xp == self.yp == 0: self.xp, self.yp = x1, y1
            #If it's black, it's eraser mode, erasing the drawn tracks; if not,
draw tracks on the image according to the selected color
            if self.Color == 'Black':
                cv.line(frame, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.eraserThickness)
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.eraserThickness)
            else:
                cv.line(frame, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.brushThickness)
                #Draw selected color lines on the created canvas
                cv.line(self.imgCanvas, (self.xp, self.yp), (x1, y1),
self.ColorList[self.Color], self.brushThickness)
                cv.circle(frame, (x1, y1), 15, self.ColorList[self.Color],
cv.FILLED)
                self.xp, self.yp = x1, y1
            else: self.xp = self.yp = 0
            #Convert the canvas color space to grayscale for subsequent image processing
imgGray = cv.cvtColor(self.imgCanvas, cv.COLOR_BGR2GRAY)
            #Perform thresholding on the grayscale image, i.e., divide the image pixel
values into two categories (e.g., foreground and background) based on the set
threshold. It is one of the core methods of image binarization.
            _, imgInv = cv.threshold(imgGray, 50, 255, cv.THRESH_BINARY_INV)
            #Convert color space, convert grayscale image to color BGR image space
imgInv = cv.cvtColor(imgInv, cv.COLOR_GRAY2BGR)
            #Perform AND and OR operations on images to combine the images
frame = cv.bitwise_and(frame, imgInv)
frame = cv.bitwise_or(frame, self.imgCanvas)
            #Draw the currently selected color box and color
            cv.rectangle(frame, (20, h - 100), (50, h - 70), self.ColorList[self.Color],
cv.FILLED)
            cv.putText(frame, self.Color, (70, h - 75), cv.FONT_HERSHEY_SIMPLEX, 0.9,
(0, 0, 255), 1)
            cv.imshow('frame', frame)

```

fingersUp finger extension detection function,

```

def fingersUp(self):
    fingers=[]

```

```

# Thumb, determine if the angle of each thumb joint is greater than 150
degrees, if so, the thumb can be considered extended
if (self.calc_angle(self.tipIds[0],
                    self.tipIds[0] - 1,
                    self.tipIds[0] - 2) > 150.0) and (
    self.calc_angle(
        self.tipIds[0] - 1,
        self.tipIds[0] - 2,
        self.tipIds[0] - 3) > 150.0): fingers.append(1)
else:
    fingers.append(0)
# 4 fingers, the other four fingers
for id in range(1, 5):
    #Here we determine if the y value of the fingertip joint is smaller than
    the middle joint, if so, the finger is extended (the origin of the image is in
    the upper left, y increases downward, x increases to the right)
    if self.lmList[self.tipIds[id]][2] < self.lmList[self.tipIds[id] - 2]
[2]:
        fingers.append(1)
    else:
        fingers.append(0)
return fingers

```

As shown in the figure below, the ID of each finger joint,



findHands hand detection function,

```

def findHands(self, frame, draw=True):
    #Create detection list to store detection results
    self.lmList = []
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing. In
    init, the self.hands object was created and initialized
    self.results = self.hands.process(img_RGB)
    if self.results.multi_hand_landmarks:
        for handLms in self.results.multi_hand_landmarks:
            if draw: self.mpDraw.draw_landmarks(frame, handLms,
self.mphand.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
            else: self.mpDraw.draw_landmarks(frame, handLms,
self.mphand.HAND_CONNECTIONS)
        #Iterate through detection results and add detection results to the
        self.lmList list, representing the ID and xy coordinates of each detected joint
        for id, lm in enumerate(self.results.multi_hand_landmarks[0].landmark):
            h, w, c = frame.shape
            cx, cy = int(lm.x * w), int(lm.y * h)

```

```
        # print(id, cx, cy)
        self.lmList.append([id, cx, cy])
    return frame, self.lmList
```